

CO2_AT3: CODING CHALLENGE

7/27/26, 9:10 AM

Student Submissions — CO2_AT3_Coding Challenge

SIMATS Engineering • CSE • CSA0804 — Python Programming • 27-07-2026 • Category: CO2_AT3_Coding Challenge • Student Submissions Report

Student 39 **Vaibhava Lakshmi K** 192511310RD

4 / 4 submitted

Q1. String Character Frequency Counter

CODE

```
text=input()
text=text.lower()
freq={}
for char in text:
    if char != ' ':
        freq[char]=freq.get(char,0)+1
for char in sorted(freq.keys()):
    print(f"{char} -> {freq[char]}")
```

INPUT

Hello World

OUTPUT

```
d -> 1
e -> 1
h -> 1
l -> 3
o -> 2
r -> 1
w -> 1
```

7/27/26, 9:10 AM

Student Submissions — CO2_AT3_Coding Challenge

SIMATS Engineering • CSE • CSA0804 — Python Programming • 27-07-2026 • Category: CO2_AT3_Coding Challenge • Student Submissions Report

Q2. Vowel and Consonant Classifier with Statistics

CODE

```
s = input().lower()
v={}
c={}
vow=0
cons=0
for ch in s:
    if ch.isalpha():
        if ch in "aeiou":
            vow+=1
            if ch in v:
                v[ch]+=1
            else:
                v[ch]=1
        else:
            cons+=1
            if ch in c:
                c[ch]+=1
            else:
                c[ch]=1
print("Vowels:",vow)
print("Consonants:",cons)
if len(v)>0:
    mv=max(v,key=v.get)
    print("Most frequent vowel:",mv,"(",v[mv],")")
else:
    print("Most frequent vowel: None")
if len(c)>0:
    lc=min(c,key=c.get)
    print("Least frequent consonant:",lc,"(",c[lc],")")
else:
    print("Least frequent consonant: None")
if cons>0:
    ratio=round(vow/cons,2)
    print("Vowel-to-consonant Ratio:",ratio)
else:
    print("Vowel-to-consonant Ratio: Undefined")
```

INPUT

Python Programming

OUTPUT

Vowels: 4
Consonants: 13
Most frequent vowel: o (2)
Least frequent consonant: y (1)
Vowel-to-consonant Ratio: 0.31

Q3. String Compression and Run-Length Encoding

CODE

```
def compress(s):
    if not s:
        return ""
    compressed=""
    count=1
    for i in range(1,len(s)):
        if s[i] == s[i-1]:
            count+=1
        else:
            compressed += s[i-1]+(str(count)if count>1 else "")
            count=1
    compressed += s[-1]+(str(count) if count>1 else "")
    if len(compressed) == len(s):
        return s
    return compressed
def decompress(s):
    if not s:
        return ""
    decompressed = ""
    i=0
    n=len(s)
    while i<n:
        char=s[i]
        i+=1
        count_str=""
        while i<n and s[i].isdigit():
            count_str+=s[i]
            i+=1
        count=int(count_str)if count_str !="" else 1
        decompressed += char*count
    return decompressed
s=input().strip()
if not s:
    print("Empty string")
else:
    compressed = compress(s)
    if compressed==s:
        print(f"For input '{s}': Compression not beneficial.")
        print(f"Original: {s}")
    else:
        decompressed=decompress(compressed)
        print(f"Compressed: {compressed}")
        print(f"Decompressed: {decompressed}")
```

INPUT

aabbccdddee
abc

OUTPUT

Compressed:a2b3c2d4e2
Decompressed:aabbccdddee

Q4. Advanced Text Statistics Analyser

CODE

```
text=input()
stop={"the","is","a","an","in","of","and","to","for"}
vowels="aeiouy"
words=["".join(c for c in w.lower() if c.isalnum()) for w in text.split() if w]
words=[w for w in words if w]
nw=len(words)
ns,np=max(1,sum(text.count(c) for c in ".!?")),max(1,len([p for p in text.split("\n\n") if p.strip()])))
syl=sum(
    max(
        1,
        sum(
            1
            for i,c in enumerate(w)
            if c in vowels and(i==0 or w[i-1] not in vowels)
        ),
    )
    for w in words
)
flesch=206.835-1.015*(nw/ns)-84.6*(syl/nw)
freq={}
for w in words:
    if w not in stop:
        freq[w]=freq.get(w,0)+1
top_5="", ".join(
    f"{w}({c})" for w,c in sorted(freq.items(),key=lambda x: (-x[1],x[0]))[:5])
print(f"Words:{nw} | Sentence:{ns} | Paragraphs:{np}")
print(f"Avg Word Length:{sum(len(w) for w in words)/nw:.1f} | Avg Sentence Length: {nw/ns:.2f}")
print(f"Top 5 Words: {top_5}\nFlesch Reading Ease:{flesch:.1f} (Standard)")
```

INPUT

Python is a powerful programming language

OUTPUT

Words:6 | Sentence:1 | Paragraphs:1
Avg Word Length:6.0 | Avg Sentence Length: 6.00
Top 5 Words: language(1, powerful(1, programming(1, python(1
Flesch Reading Ease:17.4 (Standard)